

CS 3502 Project 1: Multi-Threaded Programming

Lizbeth Ochoa

March 7, 2026

1 Phase 1 : Race Condition

1.1 Program Description

Phase 1 demonstrates race conditions using multiple teller threads in an accounting system. The teller threads perform deposits and withdrawals on a shared bank accounts with synchronization.

1.2 Program Outputs

1.3 Reflection Questions

Q1.1 - Race Condition Evidence

Run	Final total Balance
1	2130
2	1791
3	1924
4	1782
5	2069

Answer: After running the program 5 times, it is clear that there is a different total each time. The program produces a different total because multiple threads simultaneously access and modify the shared account balances. This is without synchronization. The race condition occurs when the threads read and update the same memory at the same time. The critical section occurs in these lines of code:

```
double current_balance = accounts[account_id].balance;  
accounts[account_id].balance = current_balance + amount;
```

Q1.2 Thread vs Process

Answer: A process is its own program with its own memory space and resources. While a thread is a smaller unit of execution within a process. Threads share the same memory space as other threads in the same process which is what connects to race conditions.

Q1.3 pthread_join Purpose

Answer: When I removed the pthread_join calls and ran my program, the main thread terminates and does not wait for the others. So the program becomes an incomplete output. The call is necessary because it makes sure that the main function waits for the rest of the threads to finish.

Q1.4 Non-Determinism

Answer: Non-deterministic behavior means that a program with the same inputs produces different results each time it is ran. This occurs in concurrent programs because the scheduler is what determines which thread executes first, which leads to different results. This makes debugging difficult because an error could not be easily noticed.